

Aspect	Scoped Values (Incubator) — Java 20
Java Version	20
Year	2023
Feature	Scoped Values (Incubator)
UniqueClassName	ScopedValues
Purpose	Provides a way to safely and efficiently share immutable, read-only context with methods and child threads without explicitly passing the value through every method call.
Status in Java 20	Incubator feature (jdk.incubator.concurrent)
Core idea	A value is bound to a scope and can be accessed by code executing within that scope.
Typical replacement for	ThreadLocal in situations involving contextual data, especially with structured/concurrent tasks.

Advantages and disadvantages

Category	Advantages	Disadvantages
Context propagation	Makes contextual information available deep in a call chain without passing it through every method.	The implicit context can make dependencies less obvious than explicit method parameters.
Immutability	Values are designed for safe read-only access within their scope.	If an application needs mutable per-thread state, Scoped Values are not a direct replacement.
Threading	Designed with modern concurrency models in mind and works particularly well with structured concurrency.	Java 20's implementation was still incubating, so APIs were subject to change.
Performance	Can be more efficient than using mutable ThreadLocal state for certain context-propagation scenarios.	Performance benefits depend on the workload; it should not be adopted solely for presumed performance gains.
Safety	Lexically bounded scopes	Developers must understand

	reduce the risk of accidentally modifying shared contextual state.	scope/lifetime semantics to use it correctly.
Resource management	Binding exists only for the duration of an operation, making lifecycle management simpler.	Code outside the scope cannot access the value.
Readability	Useful for cross-cutting context such as request IDs, user context, or tracing information.	Excessive use can hide important dependencies between components.
Modern concurrency	Particularly suitable for task-oriented applications where context needs to flow into child tasks.	Not useful simply because an application uses multiple threads.

When to use / when not to use

Scenario	Use Scoped Values?	Why	Example
Request correlation ID	✓ Yes	Avoids passing a request ID through many layers.	REQUEST_ID.get()
Distributed tracing context	✓ Yes	Trace information can be available throughout a request's execution.	TRACE_ID available to service methods
Authenticated user context	✓ Yes	Read-only user information can be accessed by downstream operations.	CURRENT_USER.get()
Tenant ID in multi-tenant systems	✓ Yes	Useful for propagating immutable tenant context.	TENANT_ID.get()
Locale/context information	✓ Yes	Convenient when many nested methods need the same context.	LOCALE.get()
Configuration that varies per operation	✓ Yes	Scope can define which configuration applies to an operation.	CONFIG.get()
Structured concurrent tasks	✓ Yes	Context can naturally accompany a structured task	Parent request context → child tasks

		hierarchy.	
Mutable shared state	✗ Usually no	Scoped Values are intended primarily for immutable contextual data.	Use appropriate concurrent state instead.
Data needed by only one method	✗ No	Passing a parameter is clearer and simpler.	calculate(order, currency)
Long-lived application state	✗ No	Scoped Values are associated with a bounded scope.	Use normal application configuration/state.
Caching	✗ Usually no	A cache has different lifecycle and consistency requirements.	Use ConcurrentHashMap or a caching library.
Thread-specific mutable state	✗ Usually no	ThreadLocal may still be more appropriate.	Per-thread buffers/state
Simple parameter passing	✗ No	Explicit parameters make dependencies easier to understand.	process(requestId)

Real-time use cases with examples

Real-time use case	Without Scoped Values	With Scoped Values conceptually	Benefit
Web request tracing	Pass traceId through controller → service → repository → logger.	Bind traceId once at the request boundary.	Cleaner tracing context propagation.
Microservice request context	Every method receives requestId, tenantId, etc.	Bind request context once and access it downstream.	Less parameter plumbing.
Multi-tenant SaaS	Pass tenantId	Bind the current	Reduces accidental

	through every service call.	tenant for the operation.	omission of tenant context.
Security context	Pass authenticated-user information through multiple layers.	Bind immutable user context for the operation.	Convenient read-only access.
Logging/MDC-like context	Thread-local logging context can require explicit lifecycle management.	Scoped context naturally follows the operation's lifetime.	Better lifecycle semantics.
Parallel processing	Context must be deliberately propagated to worker tasks.	Scoped Values were designed to work well with modern concurrency.	Easier contextual propagation.

Simplified example

```
import jdk.incubator.concurrent.ScopedValue;

public class ScopedValues {

    static final ScopedValue<String> REQUEST_ID =
        ScopedValue.newInstance();

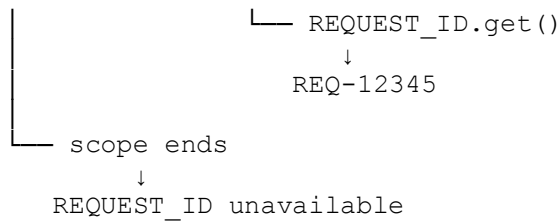
    static void service() {
        repository();
    }

    static void repository() {
        System.out.println("Request ID: " + REQUEST_ID.get());
    }

    public static void main(String[] args) {
        ScopedValue.where(REQUEST_ID, "REQ-12345")
            .run(ScopedValues::service);
    }
}
```

Conceptually:

```
main()
├── bind REQUEST_ID = "REQ-12345"
│   └── service()
│       └── repository()
│           └──
```



The important distinction is that **the value is bound to an execution scope rather than being arbitrary mutable state attached to a thread.**

Alternate features

Scoped Values (Java 20) requirement	Alternative	When the alternative is preferable	Main difference
Share contextual data across call chains	Method parameters	When only a few methods need the data	Explicit rather than implicit
Store per-thread contextual state	ThreadLocal	When mutable thread-local state is genuinely required	Mutable thread-associated state
Share immutable application-wide data	static final / immutable object	When the value does not vary by operation	Global rather than scoped
Pass request context in frameworks	Framework request context	Spring/Servlet/Micro Profile applications already providing context facilities	Framework-managed lifecycle
Propagate logging information	SLF4J/Logback MDC	When the primary requirement is logging context	Specialized for logging
Propagate distributed tracing	OpenTelemetry context	For production distributed tracing	Purpose-built tracing context
Concurrent task context	Structured concurrency + Scoped Values	Modern Java applications with hierarchical tasks	Scoped Values provide the context; structured concurrency provides task structure

--	--	--	--

Scoped Values vs ThreadLocal

Feature	Scoped Values	ThreadLocal
Introduced as	Java 20 incubator	Java 1.2
Main purpose	Immutable contextual data	Per-thread variables/state
Mutability	Read-only binding model	Mutable
Lifetime	Explicit lexical/dynamic scope	Thread-associated
Cleanup	Scope naturally ends	Often requires remove()
Child-task propagation	Designed for modern concurrency	Requires care, especially with thread pools
Accidental modification	Much harder	Possible
Best suited for	Request/operation context	Genuine thread-local state
Modern concurrency	Strong fit	Can be problematic if context is tied too closely to platform threads
API stability in Java 20	Incubator; not final	Mature/stable

Bottom line: Java 20 Scoped Values were intended to provide a **safer, more structured alternative to ThreadLocal for immutable contextual information**. They are especially valuable for request IDs, tracing, tenant/user context, and other data that should flow through an operation without being repeatedly passed as method parameters. For ordinary data dependencies or mutable thread-specific state, explicit parameters or ThreadLocal may still be the better choice.